

0.9.3. La fonction `poserS()` — poser, et passer à la ligne

`#include <poserS>` : comme `poser()`, mais la colonne 20 devient la colonne 0 de la ligne suivante.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `poserS(colonne, ligne, tuile)` pose comme `poser()`, mais une colonne **au-delà de 19** repart en colonne 0, une ligne plus bas. Avec une boucle, tout l'alphabet tient sans calcul de `% 20`.

La ligne à écrire : `#include <poserS>`.

Ses arguments : colonne, ligne, tuile, comme `poser()`.

Ce qu'elle coûte : environ **180 octets** la première fois.

Essaie : commence en colonne 10 : où tombe le Z ?

On retrouve `poserS` dans 6 leçons

[0.10](#), [0.10.1](#), [0.10.2](#), [0.86](#), [0.86.1](#), [0.86.2](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **948 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage – la mise en route de la console
main()	74 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AdresseCase	10 o	← poser() – trouve l'adresse d'une case de l'écran
Diviser	17 o	← poser() – divise – le processeur ne sait pas le faire
Reste	10 o	← poser() – le reste d'une division
CopierTuiles	18 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles – la police entière

Lettres gravées : toutes (`poserS()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>poser()</code>	× 1	49 o sur place, dans main() + AdresseCase (10 o), Diviser (17 o), Reste (10 o)
<code>image()</code>	× 1	3 o sur place, dans main() + AttendreImage (28 o)
<code>poserS()</code>	× 1	0 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

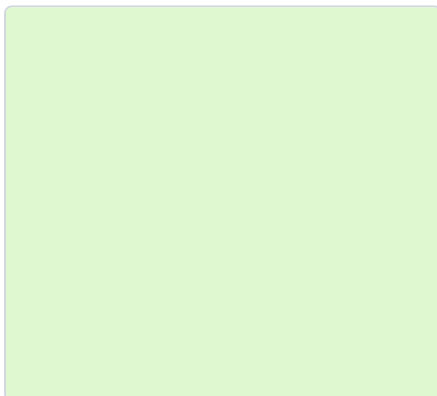
```
// ---- #include <poserS> : poser, et passer à la ligne ----  
// Les 26 lettres, de la colonne 0 à la colonne 25 : les colonnes 20 à 25  
// repartent en (0 à 5) sur la ligne d'en dessous.  
  
#include <poserS>      // poserS() : poser, et aller à la ligne au bord  
#include <ALPHABET>    // ALPHABET[i] : la lettre numéro i
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

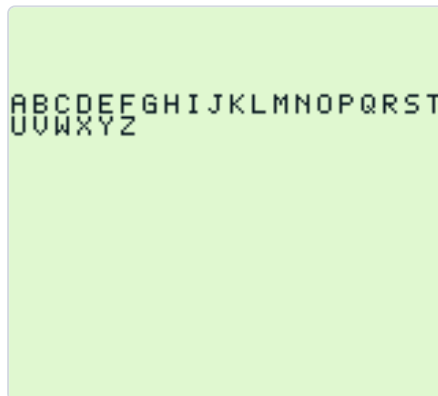
```
int main() {  
    for (uint8_t i = 0; i < 26; i++) {  
        poserS(i, 4, ALPHABET[i]); // i va de 0 à 25 : U V W X Y Z vont ligne 5  
    }  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

A à T sur la ligne 4, U à Z au début de la ligne 5.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ A à T sur la ligne 4

✓ U à Z repartent en colonne 0, ligne 5 (« UVWXYZ »)

